

Entrega Final

Asignatura: Lenguaje de Programación Java

Universidad Tecnológica Nacional - Facultad Regional
Rosario

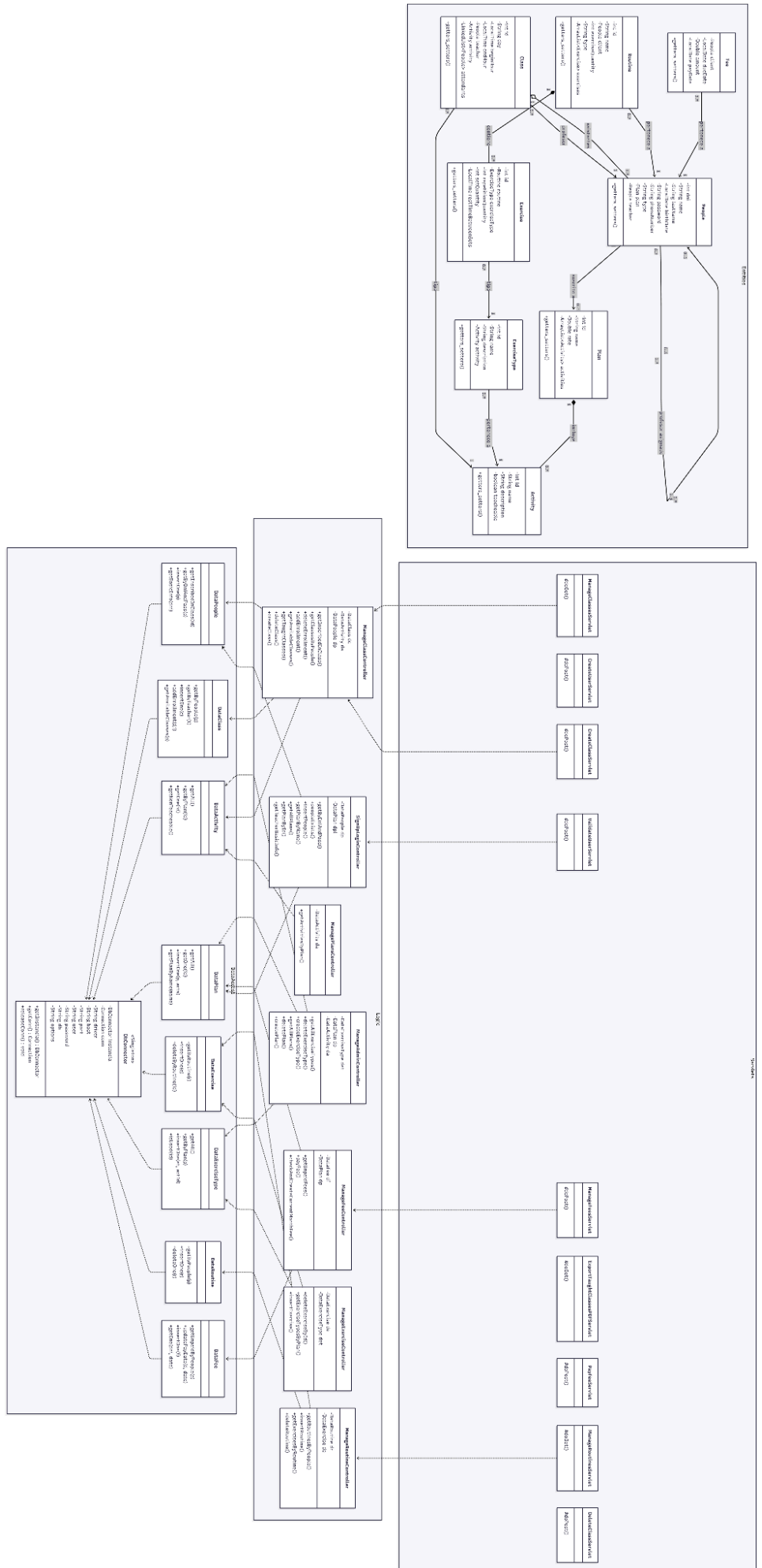
Alumno:

- Clemente Alvarez, Federico - Legajo: 49186

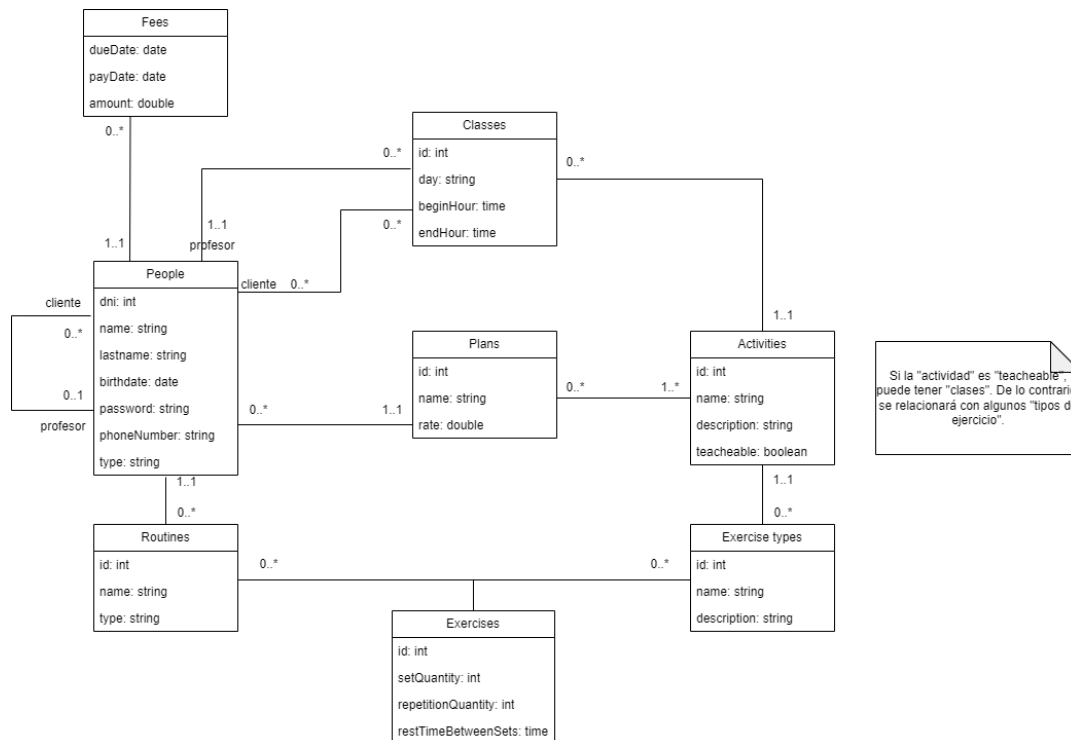
Profesor:

- Meca, Adrián

Diagrama de Clase:



Modelo de datos:



Caso de uso resumen:

Caso de Uso: Gestionar Rutinas y Ejercicios

Actor: Cliente

Precondiciones:

- El usuario está autenticado en el sistema.
- El usuario tiene un plan asignado.

Postcondiciones:

- Se registra una nueva rutina personal o se modifica una existente (añadiendo/eliminando ejercicios).
- Los ejercicios quedan vinculados a tipos de ejercicio permitidos por el plan del usuario.

Flujo principal:

1. El usuario accede a la sección "Mis Rutinas".
2. El sistema recupera y muestra la lista de rutinas asociadas al DNI del usuario.
3. El usuario selecciona la opción para crear una nueva rutina e ingresa el nombre.
4. El sistema registra la nueva rutina con tipo "Personal" y redirige al menú principal.
5. El usuario regresa a "Mis Rutinas" y selecciona la rutina recién creada para ver sus detalles.

6. El sistema muestra la lista de ejercicios de la rutina, incluyendo el detalle de cada ejercicio (nombre del tipo de ejercicio, breve descripción, series, repeticiones, tipo de descanso).
7. El usuario selecciona "Añadir Ejercicio".
8. El sistema muestra los tipos de ejercicio disponibles, filtrados según las actividades incluidas en el plan del usuario.
9. El usuario completa los datos del ejercicio (repeticiones, series, tiempo de descanso) y selecciona el tipo de ejercicio.
10. El sistema registra el ejercicio, incrementa el contador de ejercicios de la rutina y regresa a la vista de "Mis Rutinas".
11. Los pasos 5 a 10 se repiten para añadir más ejercicios.

Flujos alternativos:

- 6a. La rutina no tiene ejercicios → El sistema muestra el mensaje "Aún no tienes ejercicios".
- 7a. El usuario selecciona "Quitar" en el ejercicio → El sistema borra el ejercicio de la rutina y actualiza la vista.
- 7b. El usuario selecciona "Borrar Rutina" → El sistema borra la rutina (y por cascada sus ejercicios) y regresa a "Mis Rutinas".
- 8a. El plan del usuario no incluye actividades que tengan tipos de ejercicio relacionados → Se muestra el mensaje "Ups... Parece que tu plan no incluye ningún ejercicio".

Casos de uso de usuario sin reestructurar (sin ABMs):

Inscribirse en clases disponibles

El cliente entra a la sección de clases disponibles porque quiere anotarse en una actividad incluida en su plan. El sistema revisa qué actividades tiene habilitadas según el plan activo del usuario y muestra solamente las clases correspondientes en las que todavía no está inscripto. El usuario puede ver información de cada clase, como el día, horario, profesor y actividad. Después elige una clase y presiona el botón "Inscribirme". Entonces el sistema registra la inscripción asociando al usuario con la clase seleccionada y finalmente lo redirige a la vista de "Mis Clases", donde puede verificar que quedó anotado correctamente. Si no existen clases compatibles con su plan, el sistema informa que no hay clases disponibles. También puede decidir no inscribirse y volver al menú principal sin realizar cambios.

Dar de alta una clase (dictar clase)

El profesor accede a la sección de clases enseñadas para administrar las clases que dicta. El sistema muestra las clases que ya tiene creadas y le permite crear una nueva. Cuando selecciona la opción correspondiente, el sistema recupera todas las actividades dictables ("teacheable") y las muestra en una lista. Luego el profesor completa los datos de la nueva clase indicando la actividad, el día y el horario de inicio y finalización. Después confirma la operación y el sistema valida la información, registra la nueva clase asociándola al profesor actual y la deja disponible para que los clientes puedan inscribirse si su plan se los permite.

Finalmente el sistema vuelve a mostrar la pantalla de clases enseñadas con la nueva clase creada en la lista. Como alternativa, el profesor puede exportar en PDF todas las clases que dicta junto con la información de los usuarios inscritos en cada una.

Capturas de las pantallas:

Login / Sign Up:

Las capturas muestran tres interfaces de usuario para el sistema Genesis GYM:

- Login:** Pantalla con el logo GENESIS GYM y campos para ingresar el DNI y la Contraseña, con un botón "Ingresar".
- Sign Up:** Pantalla con el logo GENESIS GYM y el título "Ingrese sus datos personales". Incluye campos para DNI, Nombre, Apellido, Fecha de nacimiento (dd/mm/aaaa), Número de teléfono, Contraseña y DNI del profesor (si tuviese), con un botón "Continuar".
- Selección de Plan:** Pantalla con el logo GENESIS GYM y el título "Escoja un plan". Muestra dos opciones: "Premium \$1500.0/mes" (que incluye Musculacion, Pilates, Zumba, Boxeo y Musculacion pro) y "Cardio \$650.0/mes" (que incluye Pilates y Zumba).

Home Page:

Las capturas muestran la Home Page para tres tipos de usuarios:

- Federico Alvarez (Cliente independiente):** Muestra su tipo de usuario, nombre y teléfono. Incluye botones para "Mis Rutinas", "Mis Clases", "Mi Plan" y "Mis Cuotas".
- Juan Cruz Durán (Profesor):** Muestra su tipo de usuario y nombre. Incluye botones para "Mis Rutinas", "Mis Clases", "Mi Plan", "Mis Cuotas" y "Clases Enseñadas".
- Fede Admin (Administrador):** Muestra su tipo de usuario y nombre. Incluye botones para "Tipos de ejercicio" y "Planes".

Mis Rutinas:

Mis Rutinas

rutina 1

Ver

Tipo de rutina:

Personal

Cantidad de ejercicios:

0

+ Crear nueva rutina

Volver al inicio

Nueva Rutina

Nombre de la rutina

+ Crear nueva rutina

Los ejercicios podrán agregarse luego, en Mis Rutinas

rutina 1

Aún no tienes Ejercicios...

+ Agregar Ejercicio

Volver

Borrar Rutina

Nuevo Ejercicio

Ups...

Parece que tu plan no incluye ningún ejercicio

rutina 1

1)

Levantamiento de peso muerto

Quitar

El levantador eleva una barra con peso desde el suelo hasta la cintura en un movimiento fluido y compacto

Cantidad de series:

5

Repeticiones por serie:

25

Descanso entre series:

20 segs

2) Remo

Quitar

Sentado, se tracciona agarre hacia el torso activando espalda media y posterior

Cantidad de series:

5

Repeticiones por serie:

10

Descanso entre series:

15 segs

+ Agregar Ejercicio

Volver

Borrar Rutina

Nuevo Ejercicio

Cantidad de series

Cantidad de repeticiones

Tiempo entre repeticiones

segundos

Tipo de ejercicio

Dominadas

Crear ejercicio

Mis Clases:

Mis Clases

Boxeo

Cancelar

Día:

Martes

Horario:

19:30 - 20:30

Profesor:

Marcos Duke

Inscribirme a una clase

Volver al inicio

Mis Clases

Aun no estás inscrito en ninguna clase...

Inscribirme a una clase

Volver al inicio

Clases Disponibles

Pilates

Inscribirme

Día:

martes y jueves

Horario:

19:30 - 20:30

Profesor:

Juan Cruz Durán

Pilates

Inscribirme

Día:

miércoles y viernes

Horario:

17:00 - 18:00

Profesor:

Juan Cruz Durán

Zumba

Inscribirme

Día:

S7bado

Horario:

10:30 - 11:30

Profesor:

Marcos Duke

Pilates

Inscribirme

Día:

miércoles y viernes

Horario:

17:00 - 18:00

Profesor:

Juan Cruz Durán

Zumba

Inscribirme

Día:

S7bado

Horario:

10:30 - 11:30

Profesor:

Marcos Duke

Volver a Mis Clases

Clases Disponibles

No hay clases disponibles para tu plan en este momento...

Volver a Mis Clases

Mis Plan:

Mi Plan

Cardio

\$650.0/mes

Actividades que incluye:

Pilates

Zumba

Volver al inicio

Mis Cuotas:

Mis Cuotas

Vencimiento:
2026-04-30

Marcar
pagado

Monto: \$65.00

Estado: ATRASADA

Vencimiento:
2026-05-31

Marcar
pagado

Monto: \$65.00

Estado: PENDIENTE

Volver al inicio

Mis Cuotas

No tienes cuotas pendientes de pago. ¡Estás al día!

Volver al inicio

Clases Enseñadas (Profesor):

Clases Enseñadas

Pilates

Eliminar

Día: martes y jueves

Horario: 19:30 - 20:30

Pilates

Eliminar

Día: miércoles y viernes

Horario: 17:00 - 18:00

+ Crear nueva clase

Exportar a PDF

Volver al inicio

Clases Enseñadas

Aún no tienes clases asignadas como profesor.

+ Crear nueva clase

Exportar a PDF

Volver al inicio

Crear Nueva Clase

Día/s de la semana

Ej: Lunes

Hora de inicio

--|--

Hora de fin

--|--

Actividad

Pilates

Crear clase

Cancelar

Clases Enseñadas

No se puede eliminar una clase que tiene alumnos inscritos.

Pilates

Eliminar

Día: martes y jueves

Horario: 19:30 - 20:30

Pilates

Eliminar

Día: miércoles y viernes

Horario: 17:00 - 18:00

+ Crear nueva clase

Exportar a PDF

Volver al inicio

Tipos de ejercicio (Administrador):

Tipos de Ejercicio

Press-banca

Eliminar

Descripción:El levantador se tumba sobre su espalda en un banco, levantando y bajando la barra directamente por encima del pecho

Levantamiento de peso muerto

Eliminar

Descripción:El levantador eleva una barra con peso desde el suelo hasta la cintura en un movimiento fluido y compacto

Dominadas

Eliminar

Descripción:El atleta flexiona rodillas y

Cruce de poleas

Eliminar

Descripción:De pie, se aproximan brazos desde poleas laterales enfocando el trabajo en pecho

+ Nuevo tipo de ejercicio

Volver al inicio

Nuevo Tipo de Ejercicio

Nombre

Ej: Press de Banca

Descripción

Breve descripción del ejercicio

Actividad Asociada

Musculacion

Crear tipo de ejercicio

Cancelar

Planes (Administrador):

Gestión de Planes

Premium

Eliminar

Tarifa mensual: \$1500.00

Actividades incluidas:Musculacion, Pilates, Zumba, Boxeo, Musculacion pro

Cardio

Eliminar

Tarifa mensual: \$650.00

Actividades incluidas: Pilates, Zumba

Musculacion

Eliminar

Tarifa mensual: \$500.00

Actividades incluidas: Musculacion

pro

Eliminar

Tarifa mensual: \$1000.00

Actividades incluidas: Musculacion, Musculacion pro

Boxeo

Eliminar

Tarifa mensual: \$250.00

Actividades incluidas: Boxeo

+ Nuevo plan

Volver al inicio

Nuevo Plan

Nombre del Plan

Ej: Plan Musculación

Tarifa Mensual

Ej: 5000.00

Actividades Incluidas

☐ Musculacion

☐ Pilates

☐ Zumba

☐ Boxeo

☐ Musculacion pro

Crear plan

Cancelar

Fragmentos de código:

JSP:

ManageRoutines.jsp:

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <%@ page import='entities.Routine'%>
4 <%@ page import='java.util.ArrayList'%>
5 <!DOCTYPE html>
6 <html>
7 <head>
8   <meta charset="ISO-8859-1">
9   <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
10   <link
11     rel="stylesheet"
12     href="https://cdn.jsdelivr.net/npm/@picocss/pico@1/css/pico.classless.min.css"
13   />
14   <link href="ManageRoutines.css" rel="stylesheet">
15
16   <%ArrayList<Routine> routines = (ArrayList<Routine>).session.getAttribute("routines");%>
17
18   <title>Genesis Gym</title>
19 </head>
20 <body>
21   <div>
22     <article id="topCard">
23       <h1 class="title">Mis Rutinas</h1>
24     </article>
25   </div>
26   <div>
27     <%if(routines.size()==0){%>
28       <div class="noRoutines">
29         <h3>Aún no tienes rutinas...</h3>
30       </div>
31     <%}%>
32     <%if(routines.size())>0{%>
33       <for(Routine r: routines){%>
34         <div class="cardContainer">
35           <article>
36             <div class="cardTopContainer">
37               <div class="cardNameContainer">
38                 <h3><%r.getName()%></h3>
39               </div>
40               <form method="post" action="ShowRoutineServlet">
41                 <div class="cardButtonContainer">
42                   <button value=<%r.getId()%> name="routineId" type="submit">Ver</button>
43                 </div>
44               </form>
45             </div>
46             <div class="cardDataContainer">
47               <h5>Tipo de rutina: </h5>
48               <div class="pContainer">
49                 <p><%r.getType()%></p>
50               </div>
51             </div>
52             <div class="cardDataContainer" id="bottomContainer">
53               <h5>Cantidad de ejercicios: </h5>
54               <div class="pContainer">
55                 <p><%r.getExerciseQuantity()%></p>
56               </div>
57             </div>
58           </article>
59         </div>
60
61       <form method="post" action="CreateRoutineServlet">
62         <div class="buttonContainer">
63           <button type="submit"><h3 id="buttonPlus">+</h3><h5 id="buttonText">Crear nueva rutina</h5></button>
64         </div>
65       </form>
66     </div>
67
68     <div class="buttonContainer" style="margin-top: 5%;">
69       <button type="button" onclick="window.location.href='ValidateUserServlet'" style="background-color: #607d8b; border-color: #607d8b;"><h5 id="buttonText">Volver al inicio</h5></button>
70     </div>
71 </body>
72 </html>
```

CreateRoutine.jsp:

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <%@ page import='entities.Routine'%>
4 <%@ page import='java.util.ArrayList'%>
5 <!DOCTYPE html>
6 <html>
7 <head>
8   <meta charset="ISO-8859-1">
9   <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
10   <link
11     rel="stylesheet"
12     href="https://cdn.jsdelivr.net/npm/@picocss/pico@1/css/pico.classless.min.css"
13   />
14   <link href="CreateRoutine.css" rel="stylesheet">
15
16   <title>Genesis Gym</title>
17 </head>
18 <body>
19   <div>
20     <article id="topCard">
21       <h1 class="title">Nueva Rutina</h1>
22     </article>
23   </div>
24   <form method="post" action="InsertNewRoutineServlet">
25     <div class="inputContainer">
26       <label><h4>Nombre de la rutina</h4></label>
27       <input id="dni" name="routineName" type="text" required autofocus>
28     </div>
29     <div class="buttonContainer">
30       <button type="submit"><h5 id="buttonText">Crear nueva rutina</h5></button>
31     </div>
32     <p>Los ejercicios podrán agregarse luego, en Mis Rutinas</p>
33   </form>
34 </body>
35 </html>
```

ShowRoutine.jsp:

```
1  <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2  pageEncoding="ISO-8859-1"%>
3  <%@ page import='entities.Routine'%>
4  <%@ page import='entities.Exercise'%>
5  <!DOCTYPE html>
6  <html>
7  <head>
8  <meta charset="ISO-8859-1">
9  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
10 <link
11 rel="stylesheet"
12 href="https://cdn.jsdelivr.net/npm/@picocss/pico@1/css/pico.classless.min.css"
13 />
14
15 <link href="ShowRoutine.css" rel="stylesheet">
16
17 <%Routine r = (Routine) session.getAttribute("routine");
18 int i=1;%>
19
20 <title>Genesis Gym</title>
21 </head>
22 <body>
23 <div>
24 <article id="topCard">
25 <h1 class="title"><%r.getName()%></h1>
26 </article>
27 </div>
28 <div>
29 <%if(r.getExercises().size()==0){%>
30 <div class="noRoutines">
31 <h3>Aún no tienes Ejercicios...</h3>
32 </div>
33 <%}%>
34 <%if(r.getExercises().size())>0){%>
35 <%for(Exercise e: r.getExercises()){%>
36 <div class="cardContainer">
37 <article>
38 <div class="cardTopContainer">
39 <div class="cardNameContainer">
40 <h3><%i%> <%e.getExerciseType().getName()%></h3>
41 </div>
42 <form method="post" action="DeleteExerciseServlet">
43 <div class="cardButtonContainer">
44 <button value=<%e.getId()%> name="exerciseId" type="submit" id="dropButton">Quitar</button>
45 </div>
46 </form>
47 </div>
48 <div class="cardDataContainer" id="topContainer">
49 <div class="pContainer">
50 <p class="data" id="exerciseTypeDesc"><%e.getExerciseType().getDescription()%></p>
51 </div>
52 </div>
53 <div class="cardDataContainer">
54 <h5>Cantidad de series: </h5>
55 <div class="pContainer">
56 <h5 class="data"><%e.getSetQuantity()%></h5>
57 </div>
58 </div>
59 <div class="cardDataContainer">
60 <h5>Repeticiones por serie: </h5>
61 <div class="pContainer">
62 <h5 class="data"><%e.getRepetitionQuantity()%></h5>
63 </div>
64 </div>
65 <div class="cardDataContainer" id="bottomContainer">
66 <h5>Descanso entre series: </h5>
67 <div class="pContainer">
68 <h5 class="data"><%e.getRestTimeBetweenSetsInSeconds()%> segs</h5>
69 </div>
70 </div>
71 </article>
72 </div>
73 <%i++; %>
74 </div>
75 <form method="post" action="CreateExerciseServlet">
76 <div class="buttonContainer">
77 <button type="submit"><h3 id="buttonPlus">+</h3><h5 id="buttonText">Agregar Ejercicio</h5></button>
78 </div>
79 </form>
80 <div class="buttonContainer" style="margin-top: 5%;">
81 <button type="button" onclick="window.location.href='ManageRoutinesServlet'" style="background-color: #607d8b; border-color: #607d8b;">
82 <h5 id="buttonText">Volver</h5>
83 </button>
84 </div>
85 <form method="post" action="DeleteRoutineServlet">
86 <div class="buttonContainer">
87 <button type="submit" id="dropButton"><h5 id="buttonText">Borrar Rutina</h5></button>
88 </div>
89 </form>
90 </body>
91 </html>
```

CreateExercise.jsp:

```
1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <%@ page import="entities.ExerciseType"%>
4 <%@ page import="java.util.ArrayList"%>
5 <@IDOC TYPE html>
6 <html>
7 <head>
8 <meta charset="ISO-8859-1">
9 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
10 <link
11   rel="stylesheet"
12   href="https://cdn.jsdelivr.net/npm/@picocss/pico@1/css/pico.classless.min.css"
13 />
14 <link href="CreateExercise.css" rel="stylesheet">
15 <title>Genesis Gym</title>
16 <%ArrayList<ExerciseType> exTypes = (ArrayList<ExerciseType>) session.getAttribute("exerciseTypes");%>
17 </head>
18 <body>
19 <div>
20 <article id="topCard">
21 <h1 class="title">Nuevo Ejercicio</h1>
22 </article>
23 </div>
24 <%if(exTypes.size()==0) {%>
25 <div class="noPlan">
26 <article>
27 <h1 id="ups">Ups...</h1>
28 <h3>Parece que tu plan no incluye ningún ejercicio</h3>
29 </article>
30 </div>
31 <%}%>
32 <%if(exTypes.size()!=0) {%>
33 <form method="post" action="InsertNewExerciseServlet">
34 <div class="inputContainer" id="topInputContainer">
35 <label><h4>Cantidad de series</h4></label>
36 <div class="numberInputContainer">
37 <input name="setQuantity" type="number" required autofocus>
38 </div>
39 </div>
40 <div class="inputContainer">
41 <label><h4>Cantidad de repeticiones</h4></label>
42 <div class="numberInputContainer">
43 <input name="repetitionQuantity" type="number" required>
44 </div>
45 </div>
46 <div class="inputContainer">
47 <label><h4>Tiempo entre repeticiones</h4></label>
48 <div class="numberInputContainer" id="restTimeContainer">
49 <input name="restTimeBetweenSetsInSeconds" type="number" required>
50 <div id="middleDiv"></div>
51 <div id="secondsContainer">
52 <h5>segundos</h5>
53 </div>
54 </div>
55 </div>
56 <div class="inputContainer" id="selectContainer">
57 <label><h4>Tipo de ejercicio</h4></label>
58 <select name="exerciseTypeId" id="selectExType" required>
59 <%for(ExerciseType et : exTypes) {%>
60 <option value="<%=et.getId()%>"><%= et.getName()%></option>
61 <%}%>
62 </select>
63 </div>
64 </div>
65 <div class="buttonContainer">
66 <button type="submit"><h5 id="buttonText">Crear ejercicio</h5></button>
67 </div>
68 </form>
69 <%}%>
70 </body>
71 </html>
```

Servlets:

CreateRoutineServlet.java:

```
1 package servlets;
2
3 import jakarta.servlet.ServletException;
4
5 public class CreateRoutineServlet extends HttpServlet {
6   private static final long serialVersionUID = 1L;
7
8   protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
9     request.getRequestDispatcher("/WEB-INF/jsp/CreateRoutine.jsp").forward(request, response);
10   }
11 }
12
```

InsertNewRoutineServlet.java:

```
1 package servlets;
2
3 import jakarta.servlet.ServletException;
4
5 public class InsertNewRoutineServlet extends HttpServlet {
6   private static final long serialVersionUID = 1L;
7
8   protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
9     ManageRoutineController cont = new ManageRoutineController();
10     HttpSession session = request.getSession();
11     People p = (People) session.getAttribute("user");
12     Routine r = new Routine(request.getParameter("routineName"), "Personal", p);
13     cont.insertRoutine(r);
14     request.getRequestDispatcher("/WEB-INF/jsp/HomePage.jsp").forward(request, response);
15   }
16 }
17
18
19
```

ShowRoutineServlet.java:

```
1 package servlets;
2
3@import jakarta.servlet.ServletException;
14
15 public class ShowRoutineServlet extends HttpServlet {
16     private static final long serialVersionUID = 1L;
17
18     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
19         ManageRoutineController cont = new ManageRoutineController();
20         HttpSession session = request.getSession();
21
22         int routineId = Integer.parseInt(request.getParameter("routineId"));
23         ArrayList<Routine> routines = (ArrayList<Routine>) session.getAttribute("routines");
24
25         Routine r = null;
26
27         for(Routine routine : routines) {
28             if(routine.getId()==routineId) {
29                 r = routine;
30                 break;
31             }
32         }
33
34         r.setExercises(cont.getExercisesByRoutine(r));
35
36         session.setAttribute("routine", r);
37
38         request.getRequestDispatcher("/WEB-INF/jsp/ShowRoutine.jsp").forward(request, response);
39     }
40 }
41
42 }
```

DeleteExerciseServlet.java:

```
1 package servlets;
2
3@import jakarta.servlet.ServletException;
14
15 public class DeleteExerciseServlet extends HttpServlet {
16     private static final long serialVersionUID = 1L;
17
18     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
19         ManageExerciseController cont = new ManageExerciseController();
20         HttpSession session = request.getSession();
21
22         int exerciseId = Integer.parseInt(request.getParameter("exerciseId"));
23
24         cont.deleteExerciseById(exerciseId);
25
26         Routine r = (Routine) session.getAttribute("routine");
27
28         int i =0;
29         for(Exercise e: r.getExercises()) {
30             if(e.getId()==exerciseId) {
31                 break;
32             }
33             i++;
34         }
35
36         r.getExercises().remove(i);
37
38         session.setAttribute("routine", r);
39
40         request.getRequestDispatcher("/WEB-INF/jsp/ShowRoutine.jsp").forward(request, response);
41     }
42 }
43
44 }
```

CreateExerciseServlet.java:

```
1 package servlets;
2
3@import jakarta.servlet.ServletException;
16
17 public class CreateExerciseServlet extends HttpServlet {
18     private static final long serialVersionUID = 1L;
19
20     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
21         ManageExerciseController cont = new ManageExerciseController();
22         HttpSession session = request.getSession();
23
24         People p = (People) session.getAttribute("user");
25         Plan plan = p.getPlan();
26
27         ArrayList<ExerciseType> exTypes = cont.getExerciseTypesByPlan(plan);
28
29         session.setAttribute("exerciseTypes", exTypes);
30
31         request.getRequestDispatcher("/WEB-INF/jsp/CreateExercise.jsp").forward(request, response);
32     }
33 }
34
35 }
```

InsertNewExerciseServlet.java:

```
1 package servlets;
2
3 import jakarta.servlet.ServletException;
16
17
18 public class InsertNewExerciseServlet extends HttpServlet {
19     private static final long serialVersionUID = 1L;
20
21     protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
22         ManageExerciseController cont = new ManageExerciseController();
23         HttpSession session = request.getSession();
24         Routine r = (Routine) session.getAttribute("routine");
25         ExerciseType exType = new ExerciseType(Integer.parseInt(request.getParameter("exerciseTypeId")));
26
27         Exercise e = new Exercise(
28             exType,
29             Integer.parseInt(request.getParameter("repetitionQuantity")),
30             Integer.parseInt(request.getParameter("setQuantity")),
31             r
32         );
33
34         e.setRestTimeBetweenSetsInSeconds(Integer.parseInt(request.getParameter("restTimeBetweenSetsInSeconds")));
35
36         cont.insertExercise(e);
37
38         ArrayList<Routine> routines = (ArrayList<Routine>) session.getAttribute("routines");
39
40         for(Routine routine:routines) {
41             if(routine.getId()==r.getId()) {
42                 routine.setExerciseQuantity(routine.getExerciseQuantity()+1);
43                 break;
44             }
45         }
46
47         session.setAttribute("routines", routines);
48
49         request.getRequestDispatcher("/WEB-INF/jsp/ManageRoutines.jsp").forward(request, response);
50     }
51 }
52 }
```

Controllers:

ManageRoutineController.java:

```
1 package logic;
2
3 import java.util.ArrayList;
10
11 public class ManageRoutineController {
12
13     private DataRoutine dr;
14     private DataExercise de;
15
16     public ArrayList<Routine> getRoutinesByPeople(People p){
17         dr=new DataRoutine();
18         return(dr.getByPeople(p));
19     }
20
21     public void insertRoutine(Routine r) {
22         dr=new DataRoutine();
23         dr.insertOne(r);
24     }
25
26     public ArrayList<Exercise> getExercisesByRoutine(Routine r) {
27         de = new DataExercise();
28         return(de.getByRoutine(r));
29     }
30
31     public void deleteRoutine(Routine r) {
32         dr=new DataRoutine();
33         dr.deleteOne(r);
34     }
35
36 }
```

ManageExerciseController.java:

```
1 package logic;
2
3 import java.util.ArrayList;
10
11 public class ManageExerciseController {
12
13     private DataExercise de;
14     private DataExerciseType det;
15
16     public void deleteExerciseById(int exerciseId) {
17         de = new DataExercise();
18         de.deleteByRoutine(exerciseId);
19     }
20
21     public ArrayList<ExerciseType> getExerciseTypesByPlan(Plan plan){
22         det = new DataExerciseType();
23         return(det.getByPlan(plan));
24     }
25
26     public void insertExercise(Exercise e) {
27         de=new DataExercise();
28         de.insertOne(e);
29     }
30
31 }
```

DataAccess:

DataRoutine.java:

```
1 package dataAccess;
2
3 import java.sql.PreparedStatement;
4
5
6
7
8
9
10
11 public class DataRoutine {
12
13
14
15     public ArrayList<Routine> getByPeople(People p) {
16         PreparedStatement stmt = null;
17         ResultSet rs = null;
18         ArrayList<Routine> routines = new ArrayList<Routine>();
19         try {
20             stmt= DbConnector.getInstance().getConn()
21                 .prepareStatement("select r.id, r.name, r.type, count(e.id) exercise_quantity \r\n"
22                                     + "from routine r \r\n"
23                                     + "left join exercise e\r\n"
24                                     + "on e.id_routine=r.id\r\n"
25                                     + "where r.dni_people=?\r\n"
26                                     + "group by r.id");
27             stmt.setInt(1,p.getDni());
28             rs = stmt.executeQuery();
29             if (rs!=null) {
30                 while(rs.next()) {
31                     Routine r = new Routine(
32                         rs.getInt("id"),
33                         rs.getString("name"),
34                         rs.getInt("exercise_quantity"),
35                         rs.getString("type"));
36                     routines.add(r);
37                 }
38             }
39         } catch (SQLException ex){
40             ex.printStackTrace();
41         } finally {
42             try {
43                 if(rs!=null) {rs.close();}
44                 if(stmt!=null) {stmt.close();}
45                 DbConnector.getInstance().releaseConn();
46             } catch (SQLException ex) {
47                 ex.printStackTrace();
48             }
49         }
50         return(routines);
51     }
52
53
54
55
56     public void insertOne(Routine r) {
57         PreparedStatement stmt= null;
58         try {
59             stmt=DbConnector.getInstance().getConn().
60                 prepareStatement("insert into routine(name, type, dni_people) values(?,?,?)");
61
62             stmt.setString(1, r.getName());
63             stmt.setString(2, r.getType());
64             stmt.setInt(3, r.getClient().getDni());
65             stmt.executeUpdate();
66         } catch (SQLException e) {
67             e.printStackTrace();
68         } finally {
69             try {
70                 if(stmt!=null)stmt.close();
71                 DbConnector.getInstance().releaseConn();
72             } catch (SQLException e) {
73                 e.printStackTrace();
74             }
75         }
76     }
77
78     public void deleteOne(Routine r) {
79         PreparedStatement stmt= null;
80         try {
81             stmt=DbConnector.getInstance().getConn().
82                 prepareStatement("delete from routine where id=?");
83             stmt.setInt(1, r.getId());
84             stmt.executeUpdate();
85         } catch (SQLException e) {
86             e.printStackTrace();
87         } finally {
88             try {
89                 if(stmt!=null)stmt.close();
90                 DbConnector.getInstance().releaseConn();
91             } catch (SQLException e) {
92                 e.printStackTrace();
93             }
94         }
95     }
96 }
97
98 }
```

DataExercise.java:

```
1 package dataAccess;
2
3 import java.sql.PreparedStatement;
4
5 public class DataExercise {
6
7     public ArrayList<Exercise> getByRoutine(Routine r) {
8         PreparedStatement stmt = null;
9         ResultSet rs = null;
10        ArrayList<Exercise> exercises = new ArrayList<Exercise>();
11        try {
12            stmt= DbConnector.getInstancia().getConn().
13                .prepareStatement("select e.id, e.set_quantity, e.repetition_quantity, e.rest_time_between_sets, et.name et_name, et.description et_description\r\n"
14                    + "from exercise e\r\n"
15                    + "inner join exercise_type et\r\n"
16                    + "on et.id=e.id exercise_type\r\n"
17                    + "where e.id_routine=?\r\n"
18                    + "order by e.id");
19            stmt.setInt(1,r.getId());
20            rs = stmt.executeQuery();
21            if (rs!=null) {
22                while(rs.next()) {
23                    ExerciseType et = new ExerciseType(rs.getString("et_name"), rs.getString("et_description"));
24
25                    Exercise e = new Exercise(
26                        rs.getInt("id"),
27                        et,
28                        rs.getInt("repetition_quantity"),
29                        rs.getInt("set_quantity"),
30                        rs.getObject("rest_time_between_sets",LocalTime.class));
31
32                    exercises.add(e);
33                }
34            }
35        } catch (SQLException ex){
36            ex.printStackTrace();
37        } finally {
38            try {
39                if(rs!=null) {rs.close();}
40                if(stmt!=null) {stmt.close();}
41                DbConnector.getInstancia().releaseConn();
42            } catch (SQLException ex) {
43                ex.printStackTrace();
44            }
45        }
46        return(exercises);
47    }
48
49    public void deleteByRoutine(int exerciseId) {
50        PreparedStatement stmt= null;
51        try {
52            stmt=DbConnector.getInstancia().getConn().
53                prepareStatement("delete from exercise where id=?");
54            stmt.setInt(1, exerciseId);
55            stmt.executeUpdate();
56        } catch (SQLException e) {
57            e.printStackTrace();
58        } finally {
59            try {
60                if(stmt!=null)stmt.close();
61                DbConnector.getInstancia().releaseConn();
62            } catch (SQLException e) {
63                e.printStackTrace();
64            }
65        }
66    }
67
68    public void insertOne(Exercise ex) {
69        PreparedStatement stmt= null;
70        try {
71            stmt=DbConnector.getInstancia().getConn().
72                prepareStatement("insert into exercise \r\n"
73                    + "(id_routine, id_exercise_type, set_quantity, repetition_quantity, rest_time_between_sets)\r\n"
74                    + "values (?, ?, ?, ?, ?)");
75            stmt.setInt(1, ex.getRoutine().getId());
76            stmt.setInt(2, ex.getExerciseType().getId());
77            stmt.setInt(3, ex.getSetQuantity());
78            stmt.setInt(4, ex.getRepetitionQuantity());
79            stmt.setObject(5, ex.getRestTimeBetweenSets());
80            stmt.executeUpdate();
81        } catch (SQLException e) {
82            e.printStackTrace();
83        } finally {
84            try {
85                if(stmt!=null)stmt.close();
86                DbConnector.getInstancia().releaseConn();
87            } catch (SQLException e) {
88                e.printStackTrace();
89            }
90        }
91    }
92
93 }
94
95 }
```


Entities:

Routine.java:

```
1 package entities;
2
3 import java.util.ArrayList;
4
5 public class Routine {
6
7     private int id;
8     private String name;
9     private People client;
10    private int exerciseQuantity;
11    private String type;
12    private ArrayList<Exercise> exercises;
13
14
15    public Routine(int id, String name, int exerciseQuantity, String type) {
16        this.name = name;
17        this.type = type;
18        this.id = id;
19        this.exerciseQuantity = exerciseQuantity;
20        exercises = new ArrayList<Exercise>();
21    }
22
23    public Routine(String name, String type, People client) {
24        this.name = name;
25        this.type = type;
26        this.client = client;
27        exercises = new ArrayList<Exercise>();
28    }
29
30    public String getName() {
31        return name;
32    }
33    public void setName(String name) {
34        this.name = name;
35    }
36    public People getClient() {
37        return client;
38    }
39    public void setClient(People client) {
40        this.client = client;
41    }
42
43    public String getType() {
44        return type;
45    }
46    public void setType(String type) {
47        this.type = type;
48    }
49    public int getId() {
50        return id;
51    }
52    public void setId(int id) {
53        this.id = id;
54    }
55    public int getExerciseQuantity() {
56        return exerciseQuantity;
57    }
58    public void setExerciseQuantity(int exerciseQuantity) {
59        this.exerciseQuantity = exerciseQuantity;
60    }
61
62    public ArrayList<Exercise> getExercises() {
63        return exercises;
64    }
65
66    public void setExercises(ArrayList<Exercise> exercises) {
67        this.exercises = exercises;
68    }
69
70
71 }
```

Exercise.java:

```
1 package entities;
2 import java.time.LocalDateTime;
3
4 public class Exercise {
5
6     private int id;
7     private Routine routine;
8     private ExerciseType exerciseType;
9     private int repetitionQuantity;
10    private int setQuantity;
11    private LocalDateTime restTimeBetweenSets;
12
13
14    public Exercise(int id, ExerciseType exerciseType, int repetitionQuantity, int setQuantity, LocalDateTime restTimeBetweenSets) {
15        this.id = id;
16        this.exerciseType = exerciseType;
17        this.repetitionQuantity = repetitionQuantity;
18        this.setQuantity = setQuantity;
19        this.restTimeBetweenSets = restTimeBetweenSets;
20    }
21
22    public Exercise(ExerciseType exerciseType, int repetitionQuantity, int setQuantity, Routine routine) {
23        this.exerciseType = exerciseType;
24        this.repetitionQuantity = repetitionQuantity;
25        this.setQuantity = setQuantity;
26        this.routine = routine;
27    }
28
29    public Routine getRoutine() {
30        return routine;
31    }
32    public void setRoutine(Routine routine) {
33        this.routine = routine;
34    }
35    public ExerciseType getExerciseType() {
36        return exerciseType;
37    }
38    public void setExerciseType(ExerciseType exerciseType) {
39        this.exerciseType = exerciseType;
40    }
41    public int getRepetitionQuantity() {
42        return repetitionQuantity;
43    }
44    public void setRepetitionQuantity(int repetitionQuantity) {
45        this.repetitionQuantity = repetitionQuantity;
46    }
47    public int getSetQuantity() {
48        return setQuantity;
49    }
50    public void setSetQuantity(int setQuantity) {
51        this.setQuantity = setQuantity;
52    }
53    public LocalDateTime getRestTimeBetweenSets() {
54        return restTimeBetweenSets;
55    }
56    public void setRestTimeBetweenSets(LocalDateTime restTimeBetweenSets) {
57        this.restTimeBetweenSets = restTimeBetweenSets;
58    }
59    public int getId() {
60        return id;
61    }
62    public void setId(int id) {
63        this.id = id;
64    }
65
66    public int getRestTimeBetweenSetsInSeconds() {
67        String[] timeArray = this.getRestTimeBetweenSets().toString().split(":");
68        int seconds = Integer.parseInt(timeArray[0])*60+Integer.parseInt(timeArray[1])*60+Integer.parseInt(timeArray[2]);
69        return seconds;
70    }
71
72    public void setRestTimeBetweenSetsInSeconds(int secondsInput) {
73        int hours = secondsInput/3600;
74        int minutes = (secondsInput-hours*3600)/60;
75        int seconds = (secondsInput-hours*3600-minutes*60);
76
77        String hoursString = Integer.toString(hours);
78        String minutesString = Integer.toString(minutes);
79        String secondsString = Integer.toString(seconds);
80
81        if(hours<10) {
82            hoursString = "0"+hoursString;
83        }
84        if(minutes<10) {
85            minutesString = "0"+minutesString;
86        }
87        if(seconds<10) {
88            secondsString = "0"+secondsString;
89        }
90
91        String timeString = hoursString+": "+minutesString+": "+secondsString;
92        this.restTimeBetweenSets = LocalDateTime.parse(timeString);
93    }
94
95
96 }
```